DECIMO AÑO · DISEÑO WEB

Ingeniería del Software

Importancia y Modelos de Procesos en el Desarrollo Web

```
function buildWeb() {
  plan();
  design();
  develop();
  test();
  deploy();
}
```

Agenda de la Leccion

01

¿Que es la Ingenieria del Software?

Definicion IEEE, 3 pilares y por que existe como disciplina

02

¿Por que importa en la Web?

Consecuencias reales de construir sin metodologia

03

Modelos de Proceso en profundidad

Cascada, Incremental y Agil — fases, ejemplos, ventajas y limitaciones

04

Aplicacion en proyecto web real

Como elegir el modelo correcto segun el tipo de proyecto

¿Que es la Ingenieria del Software?

"La aplicacion de un enfoque sistematico, disciplinado y cuantificable al desarrollo, operacion y mantenimiento del software." — IEEE, 1990

📐 Sistematico

Seguir pasos ordenados y planeados. No improvisar. Tener un plan desde el inicio.

📏 Disciplinado

Cumplir reglas, estandares y buenas practicas.
Todo el equipo sigue las mismas normas.

📊 Cuantificable

Medir tiempos, costos y calidad con datos.
Ejemplo: cuantos bugs hay, cuanto tarda cada fase.

El Problema: ¿Que pasa SIN Ingenieria del Software?

❌ SIN Ingenieria

- Codigo desorganizado, dificil de entender
- Nadie sabe en que trabaja el otro
- El sitio falla cuando mas lo necesitan
- Los cambios rompen otras partes del sistema
- El proyecto se atrasa semanas o meses
- El cliente rechaza el producto final

✅ CON Ingenieria

- Codigo claro, documentado y reutilizable
- Cada persona conoce su rol y sus tareas
- El sitio funciona de forma estable
- Los cambios se gestionan sin romper nada
- El proyecto se entrega en el tiempo pactado
- El cliente recibe exactamente lo que pidio

¿Por que importa en el Desarrollo Web?

Sitios mas confiables

Un proceso bien definido reduce errores en produccion que afectan a miles de usuarios al mismo tiempo.

Desarrollo mas rapido

Los modelos de proceso ayudan a dividir el trabajo para que varios desarrolladores trabajen en paralelo.

Mayor seguridad

La planeacion previa incluye etapas de prueba de seguridad antes de publicar el sitio al publico.

Ahorro de costos

Detectar errores en planificacion es 100 veces mas barato que corregirlos cuando el sitio ya esta publicado.

Trabajo en equipo

Los roles y procesos claros permiten que varios desarrolladores colaboren sin pisarse el trabajo.

Escalabilidad

Un sitio bien construido puede crecer, agregar secciones y nuevas funciones sin rehacer todo desde cero.

Los Modelos de Proceso: ¿Que son y cuales existen?

Un modelo de proceso define el ORDEN y la FORMA en que se realizan las actividades para crear software. Actua como una hoja de ruta: le dice al equipo que hacer primero, quien lo hace y como saber que esa etapa esta terminada.

Requisitos → **Diseno** → **Desarrollo** → **Pruebas** → **Despliegue** → **Mantenimiento**

Estas 6 actividades son comunes a todos los modelos. Lo que cambia es COMO se organizan:

Cascada

Lineal

Las 6 fases se ejecutan UNA SOLA VEZ, en orden estricto. No se avanza hasta completar la fase anterior.

Incremental

Modular

Las 6 fases se repiten por MODULOS. Cada modulo entrega una parte funcional al cliente.

Agil

Iterativo

Las 6 fases se repiten en CICLOS CORTOS (sprints). El cliente valida al final de cada ciclo.

Modelo en Cascada — Las 5 Fases en Detalle

Origen: Winston Royce, 1970 · Regla de oro: una fase termina al 100% antes de iniciar la siguiente.

⚠️ Si se omite o falla una fase, las siguientes heredan ese error.

1	REQUISITOS	**QUE: ¿QUE debe hacer el sitio?** Se reune con el cliente para documentar TODAS las funciones, paginas y...	📤 Documento de requisitos firmado por el cliente.
2	DISENO	**QUE: ¿COMO sera construido?** Se definen la arquitectura del sistema, la base de datos, el aspecto v...	📤 Planos del sistema: diagramas, maquetas y estructura de carpetas.
3	DESARROLLO	**QUE: Escribir el codigo** Los programadores implementan todo lo disenado. HTML, CSS, JS, backend...	📤 Codigo fuente completo de todas las paginas y funciones.
4	PRUEBAS	**QUE: ¿Todo funciona?** Un equipo de QA prueba el sistema completo: funcionalidad, rendimiento...	📤 Reporte de bugs corregidos. Sitio aprobado para publicar.
5	DESPLIEGUE Y MANTENIMIENTO	**QUE: Lanzar y mantener** El sitio se sube al servidor de produccion. Luego se monitorea, actual...	📤 Sitio en linea. Plan de mantenimiento activo.

Cascada — Ventajas, Limitaciones y Ejemplo Real

✅ Ventajas

- **Facil de entender**
  Cualquier persona puede seguir el flujo de fases.
- **Documentacion robusta**
  Cada fase produce documentos que guian la siguiente.
- **Progreso medible**
  Se puede saber exactamente en que fase esta el equipo.
- **Ideal para contratos fijos**
  Cuando precio y alcance no cambian, funciona perfecto.
- **Bueno para principiantes**
  Los roles y responsabilidades son muy claros.

❌ Limitaciones

- **Cero flexibilidad**
  Un cambio de requisitos obliga a reiniciar fases.
- **Cliente ve el resultado tarde**
  No hay entregas parciales; todo al final.
- **Errores tempranos son caros**
  Un error en fase 1 se arrastra a todas las demas.
- **Poco realista hoy en dia**
  Los proyectos modernos cambian constantemente.
- **No apto para proyectos largos**
  A mayor duracion, mayor riesgo de fracaso total.

🏪 Ejemplo ideal para Cascada:

El MEP encarga un sitio informativo de 4 paginas fijas (inicio, sobre nosotros, contacto, noticias), con presupuesto cerrado de ₡500.000 y sin cambios esperados. Cascada es perfecto: los requisitos son claros, el equipo sabe exactamente que construir.

Modelo Incremental — Concepto y Funcionamiento

Concepto clave

En lugar de entregar TODO al final, el software se divide en MODULOS funcionales. Cada modulo pasa por sus propias fases: requisitos → diseno → desarrollo → pruebas. Resultado: el cliente puede usar partes del sistema mientras las demas se construyen.

Origen: Barry Boehm y Harlan Mills, 1980s

Nacio porque Cascada fallaba en proyectos medianos donde el cliente queria ver avances antes de que todo estuviera listo. La clave: cada incremento ES funcional y util por si solo.

Incremento 1

Semana 1-4 → MVP (minimo viable)

Req → Diseño → Dev → Test → ✅ Entrega

- Pagina de inicio con diseno del colegio
- Menu de navegacion funcional
- Version responsive para celular

Incremento 2

Semana 5-8 → Funciones sociales

Req → Diseño → Dev → Test → ✅ Entrega

- Seccion de noticias con editor
- Galeria de fotos de eventos
- Formulario de contacto con email

Incremento 3

Semana 9-12 → Administracion

Req → Diseño → Dev → Test → ✅ Entrega

- Panel de administracion con login
- Subir y editar contenido sin codigo
- Estadisticas de visitas del sitio

Incremental — Ventajas, Limitaciones y Ejemplo Real

✅ Ventajas

- **El cliente usa el sistema desde antes**
  No espera al final para tener algo funcional.
- **Errores se detectan por modulo**
  Un fallo en el Incremento 2 no afecta el 1.
- **Retroalimentacion temprana**
  El cliente puede pedir ajustes entre incrementos.
- **Reduce el riesgo total del proyecto**
  Si falla un modulo no fracasa todo el trabajo.
- **Mas facil de probar**
  Cada modulo pequeno es mas facil de revisar.

❌ Limitaciones

- **Planificacion inicial cuidadosa**
  Hay que dividir bien el sistema desde el inicio.
- **Integracion puede ser compleja**
  Unir todos los modulos al final requiere trabajo.
- **Documentacion puede quedar incompleta**
  Cada modulo se documenta por separado.
- **No ideal si los requisitos cambian mucho**
  El plan de modulos se puede desalinear.
- **Requiere disciplina del equipo**
  Cada modulo debe estar 100% terminado y probado.

🛒 Ejemplo ideal para Incremental:

Una tienda en linea que empieza con catalogo y carrito (Incremento 1), luego agrega pasarela de pagos (Incremento 2) y finalmente cuenta de usuario con historial (Incremento 3). El negocio ya vende desde el Incremento 1.

Metodologias Agiles — Manifiesto y 4 Valores Fundamentales

En 2001, 17 expertos en software se reunieron en Utah (EE.UU.) y redactaron el Manifiesto Agil. Su objetivo: crear software de forma mas humana, colaborativa y adaptable al cambio, superando la rigidez de modelos como Cascada.

Los 4 Valores del Manifiesto Agil:

MAS IMPORTANTE:
Individuos e interacciones

sobre:
procesos y herramientas

Las personas y su comunicacion diaria son mas importantes que seguir un proceso rigido o usar herramientas sofisticadas.

MAS IMPORTANTE:
Software funcionando

sobre:
documentacion extensa

Entregar un programa que funcione vale mas que producir cientos de paginas de documentacion que nadie lee.

MAS IMPORTANTE:
Colaboracion con el cliente

sobre:
negociacion de contratos

Trabajar de la mano con el cliente durante todo el proyecto es mas valioso que acordar todo al inicio y no verlo mas.

MAS IMPORTANTE:
Responder al cambio

sobre:
seguir un plan rigido

Adaptarse a los cambios del cliente o del mercado tiene mas valor que seguir al pie de la letra el plan original.

Metodologias Agiles — El Sprint: El Motor de Agil

Un SPRINT es el ciclo basico de Agil (1-4 semanas). Al final de cada sprint se entrega al cliente algo que FUNCIONA para que lo revise.

El ciclo de un Sprint:

1. Planificar Sprint

¿Que tareas completamos esta semana?

2. Desarrollar y Probar

El equipo codifica y prueba diariamente.

🔄 SPRINT 1-4 sem.

3. Revisar con el Cliente

Se muestra lo construido al cliente.

4. Adaptar y Mejorar

Se ajusta el plan para el proximo sprint.

Frameworks Agiles principales:

🏉 SCRUM

- Sprints de 1-4 semanas con objetivos fijos
- Roles: Product Owner, Scrum Master, Dev Team
- Daily Standup: reunion de 15 min cada dia

📋 KANBAN

- Tablero visual: Por hacer / En progreso / Listo
- Sin sprints fijos — flujo continuo de trabajo
- Se limita el trabajo simultaneo (WIP limit)

⚡ XP (Extreme Programming)

- Programacion en pares: 2 personas, 1 computadora
- Pruebas automatizadas antes de escribir codigo
- Entregas muy frecuentes (cada pocos dias)

Agil — Ventajas, Limitaciones y Ejemplo Real

✅ Ventajas

- **Alta flexibilidad**
  Se puede cambiar la direccion en cualquier sprint.
- **Entregas de valor constantes**
  El cliente recibe funcionalidad cada 1-4 semanas.
- **Problemas se detectan rapido**
  Los errores se encuentran en el mismo sprint.
- **El cliente es parte del equipo**
  Valida y da retroalimentacion en cada ciclo.
- **Equipos mas motivados**
  La autonomia y los logros frecuentes motivan al equipo.

❌ Limitaciones

- **Requiere cliente disponible**
  Si el cliente no da retroalimentacion, el sprint falla.
- **Dificil escalar a equipos grandes**
  Mas de 9 personas por equipo genera caos.
- **Documentacion minima**
  Puede causar problemas si el equipo cambia.
- **Scope creep**
  El alcance puede crecer sin control sprint a sprint.
- **No apto para requisitos 100% fijos**
  En contratos cerrados no hay margen para cambios.

🚀 Ejemplo ideal para Agil:

Un startup que quiere crear una app de delivery para su ciudad. No saben exactamente que funciones necesitaran. Con Agil, lanzan en 2 semanas con registro y pedidos basicos, y van agregando funciones (seguimiento en tiempo real, calificaciones, historial) segun lo que los usuarios pidan.

Comparando los 3 Modelos — Tabla Resumen

Caracteristica	🌊 Cascada	📦 Incremental	🔄 Agil
Flujo de trabajo	Lineal — una fase completa a la vez	Por modulos funcionales separados	Ciclos cortos (sprints de 1-4 sem)
Flexibilidad	Muy baja. Cambios = reiniciar fases	Media. Se ajusta entre incrementos	Muy alta. Cambia cada sprint
Participacion del cliente	Solo al inicio y al final	Al entregar cada incremento	Constante — revisa cada sprint
Cuando se ve el producto	Al terminar el proyecto completo	Al final de cada incremento	Al final de cada sprint
Riesgo de fracaso	Alto si requisitos cambian	Medio — modulos independientes	Bajo — se corrige sprint a sprint
Documentacion	Muy detallada y formal	Moderada por modulo	Minima — se priorizan entregas
Mejor para...	Proyectos pequenos y fijos	Proyectos medianos bien definidos	Proyectos grandes o cambiantes

¿Como Elegir el Modelo Correcto para tu Proyecto?

Hazte estas 3 preguntas antes de elegir:

❓ ¿Los requisitos estan 100% claros y NO cambiaran?

SI → 🌊 Cascada: documenta todo y ejecuta en orden.

NO → 📦 Incremental o 🔄 Agil para adaptarse.

❓ ¿El cliente necesita ver avances mientras se construye?

SI → 📦 Incremental (modulos) o 🔄 Agil (sprints).

NO → 🌊 Cascada si puede esperar el producto final.

❓ ¿El proyecto durara mas de 3 meses y/o tiene mas de 5 personas?

SI → 🔄 Agil: equipos y proyectos grandes necesitan iteracion.

NO → 🌊 Cascada o 📦 Incremental para proyectos cortos.

Aplicacion en Proyecto Real: Sitio Web del Colegio

Escenario: El colegio necesita un sitio web para noticias, horario y contactar a profesores. Equipo de 3 estudiantes, 3 meses de plazo.

Modelo elegido: 📦 Incremental — plazo definido, funcionalidades claras, pero el director quiere ver avances

Requisitos	Se reune con el director para listar: paginas necesarias, colores del colegio, quienes pueden editar contenido.
Diseno	Se hacen bocetos en papel de cada pagina. Se define navegacion: menu, footer, paleta de colores institucional.
Desarrollo	Sprint 1: Home + Menu. Sprint 2: Noticias. Sprint 3: Horario. Sprint 4: Contacto + Panel de admin.
Pruebas	Se prueba en Chrome, Firefox, Safari, en celular y tablet. Se valida velocidad de carga y formularios.
Despliegue	Se sube a hosting con dominio del colegio. Se capacita al encargado de noticias para actualizar contenido.

LO QUE APRENDIMOS HOY

La Ingenieria del Software da sistematizacion y orden al desarrollo.
Los 3 modelos de proceso son guias con diferente nivel de flexibilidad y estructura.
Elegir el modelo correcto depende del proyecto, el cliente y los requisitos.

Cascada

Lineal · Fijo
Una fase a la vez

Incremental

Modular · Parcial
Entregas por partes

Agil

Iterativo · Flexible
Sprints frecuentes